TECHNICAL SOLUTION

UI-test 智能测试体系技术方案

面向技术负责人，聚焦 UI 自动化分层、回归效率、视觉治理与 Azure 扩展路径

5 个测试代理

3 条日常测试链路

Azure / VLM 可选增强

方案核心

- 以 ui-test 作为统一入口，对测试请求做快速分类与路由。
- 以 discovery 复用环境、路由、组件、定位器、旅程等上下文。
- 以 CT、E2E、Visual 三条链路承担日常测试执行。
- 以 Governance 独立承接 Azure、CI/CD、VLM 与基线策略。
- 默认本地优先，治理能力按需升级，避免一开始就引入重流程。

基于当前 UI-test-Demo 仓库现状梳理

EXECUTIVE SUMMARY

为什么现在要升级 UI 测试体系

核心目标不是“多写测试”，而是把质量验证做成可编排、可治理、可扩展的工程能力。

统一入口

降低协作成本

测试诉求统一进入 ui-test，再由专业代理接管，避免“先问工具、后问目标”的沟通摩擦。

复用 Discovery

减少重复分析

路由、组件、i18n、locator、journey 不再被每条链路重复发现，降低生成测试时的随机性。

治理可插拔

本地优先，云上增强

本地默认使用原生 Playwright，只有在需要 PR 门禁、Azure 云执行、VLM 审查时再引入治理链路。

1 质量目标

发现回归更早、定位更快。

2 工程目标

测试链路标准化，可重复生成。

3 治理目标

基线与云执行能力可控引入。

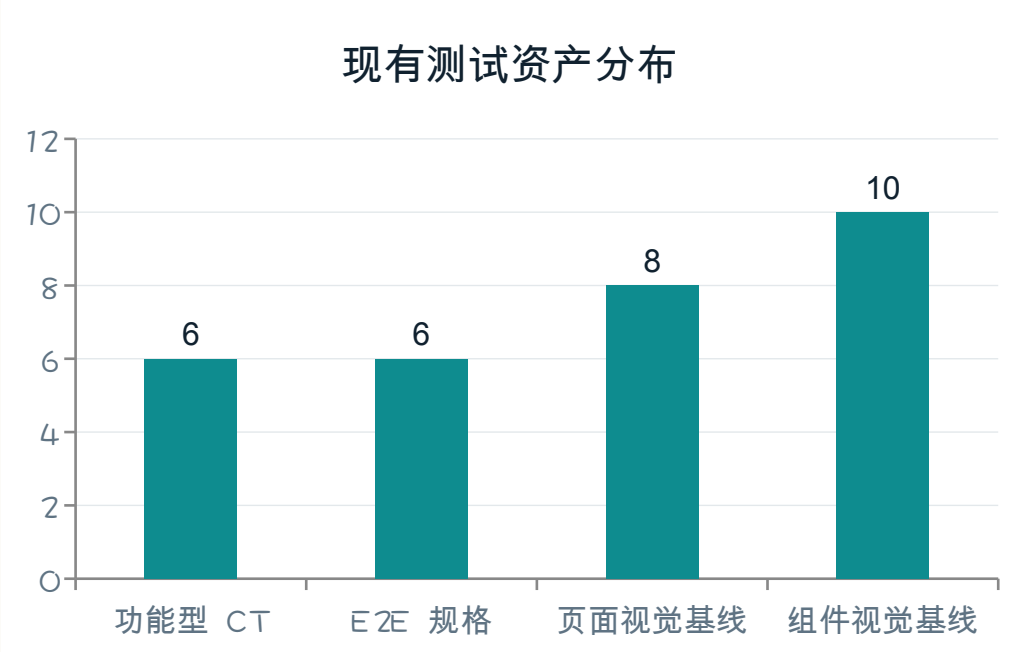
4 业务目标

保障关键页面和用户旅程稳定。

CURRENT STATE

仓库现状与机会点

当前项目已经具备较好的 Playwright 基础，但覆盖结构仍偏“点状”，需要升级为体系化能力。



关键观察

- 日常执行已分为 E2E 、 Visual 、 CT 三条本地链路，基础配置完整。
- 视觉测试具备 VLM fallback 扩展，但默认仍保持像素比对优先。
- Trip/Favorites 等业务页面已有视觉基线，功能型 E2E 仍相对薄弱。
- 组件测试重点集中于 Navbar 、 DestinationCard 、 FilterBar 、 SearchBar ，仍有组件只做了视觉基线。

优势：配置与目录结构已经稳定

适合在现有基础上升级，不需要重做框架。

缺口：功能覆盖与视觉覆盖不均衡

需要一套更清晰的场景矩阵与推进优先级。

机会：治理能力可以按需接入

适合把 Azure / CI / VLM 留在后续治理阶段。

ARCHITECTURE

总体架构：轻入口 + 专业代理 + 可复用发现层

把“请求分类、测试执行、云上治理”拆开，是这套体系能长期演进的关键。

ui-test 统一入口

接收自然语言测试需求，判断应走哪一条能力链路。

Discovery

环境、路由、组件、locator、journey

Component

隔离渲染、props、event、provider

E2E

页面流、筛选、导航、用户旅程

Visual

页面 / 组件基线、差异分析

Governance

Azure、CI、VLM、基线策略

配置与运行层

playwright.config.ts / playwright-ct.config.ts / playwright.service.config.ts

执行产物层

HTML 报告、test-results、视觉快照、VLM JSON、Azure Portal 结果

组织收益层

统一协作入口、回归效率、基线治理、云执行能力、面向 PR 的质量门禁

SCENARIO MATRIX

测试场景矩阵

不同链路解决的是不同层次的问题，不能用一套测试去替代全部需求。

链路	核心目标	主要输入	主要输出
Component / CT	验证组件渲染、 props 、 event 、条件分支	组件路径、样例 props 、 provider 依赖、关键渲染输出	组件 spec 、挂载断言、失败定位结论
E2E	验证页面流程、交互与用户旅程	路由清单、交互点、 locator 建议、预期 URL / 文本 / 状态变化	页面 spec 、旅程 spec 、失败证据与 root cause 分析
Visual	验证 UI 布局与视觉回归	页面清单、视口、状态变体、基线环境、 i18n 变体	基线快照、差异截图、像素或语义级判定
Governance	定义 Azure / CI / VLM / 基线治理规则	套件目标、 Workspace 、认证方式、门禁策略、成本约束	执行策略、 CI 约束、审查标准、平台报表

原则 1 ：先问测试目标，再决定链路。

原则 2 ：先像素、后 VLM ，保持默认链路简单。

原则 3 ： CT 不上 Azure 。

PROCESS FLOW

端到端测试流程设计

把“从需求到报告”的每一步标准化，才能把测试从手工作坊变成稳定流程。

需求进入
自然语言需求进入 ui-test

能力分类
路由到 discovery / CT / E2E / Visual / Governance

上下文补齐
补齐路由、组件、locator、journey、环境

测试执行
运行 Playwright 本地或 Azure 云上套件

结果沉淀
产出 HTML 报告、截图、JSON、Portal 记录

标准化检查点

- 输入不完整时，优先走 discovery，而不是直接猜测组件或 locator。
- 测试生成后默认先本地执行；只有在用户明确提出 CI / Azure / 门禁要求时才升级治理链路。
- 失败处理遵循 report-only：定位、分类、给方向，不默认自动改业务代码。
- 视觉回归默认使用原生 toHaveScreenshot，只有像素失败且 VLM_REVIEW=true 时才触发语义判断。

INPUT / OUTPUT

输入输出契约

对技术负责人而言，最重要的不是“有多少测试”，而是每一条链路的输入是否稳定、输出是否可消费。

输入层

- 测试意图：组件、E2E、视觉、Azure、VLM、CI 门禁。
- 发现产物：Project Analysis Report、Route Inventory、Component Inventory、Locator Notes、Journey Notes。
- 运行参数：baseURL、视口、语言、样例数据、localStorage seed、云端认证。
- 治理约束：基线环境、并发、成本上限、PR 通过条件、VLM 阈值。

输入设计的目标：减少猜测，提升生成稳定性。

输出层

- 代码层：CT/E2E/Visual spec、fixture、reporter、service config。
- 证据层：HTML 报告、test-results、trace、失败截图、视觉快照。
- 治理层：vlm-review-report.json、Azure Portal Test Runs、CI Artifact。
- 决策层：通过 / 失败 / 基线更新 / 低置信度待人工复核。

输出设计的目标：让测试结果能直接被研发、测试、CI 消费。

REPO MAPPING

当前仓库落地映射

技术方案不是从零开始，而是建立在当前已经可运行的仓库资产之上。

配置与脚本

- playwright.config.ts ：本地 E2E + Visual 。
- playwright-ct.config.ts ：本地组件测试。
- playwright.service.config.ts ： Azure Playwright Workspace 。
- package.json 脚本： test:e2e / test:ct / test:visual / test:azure:* 。

测试目录

- tests/component ：功能型组件测试 + 组件视觉基线。
- tests/e2e ：页面验证、导航、i18n 、用户旅程。
- tests/visual ：页面级视觉回归。
- tests/fixtures 与 tests/utils ： wrapper 、 VLM fixture 、 reporter 、 reviewer 。

建议一：优先补 **Trip / Favorites** 的功能型 **E2E**。

已有业务覆盖

- 首页、列表页、详情页、About 已有 E2E 。
- Favorites 、 Trips 、 TripEdit 已有视觉基线。
- Navbar 、 DestinationCard 、 FilterBar 、 SearchBar 已有核心 CT 。

建议二：对仅有视觉基线的组件补充行为断言。

GOVERNANCE

治理与平台扩展边界

默认把日常测试留在本地，把复杂性留给真正需要治理的场景。

Local First

默认执行模型

日常 E2E、Visual、CT 都优先使用本地 Playwright。
优点是反馈快、链路简单、对团队门槛低。

Azure PT

云执行与规模化

只承接 E2E 与 Visual。CT 不上云。
适合 PR 门禁、统一浏览器环境、并行执行、平台可观测。

VLM Review

视觉语义兜底

仅在像素差异失败且显式开启 VLM_REVIEW=true 时触发。
目的不是替代像素比对，而是降低误报、辅助人工判断。

治理边界结论：把 Azure、CI、VLM 视为“可插拔能力”，而不是所有团队一开始就必须承受的复杂度。

ROADMAP

分阶段实施路线

先补覆盖，再补治理；先把高频问题解决，再把平台能力做深。

Phase 1
覆盖补齐

- 补 Trip / Favorites / TripEdit 的 E2E 关键路径
- 补 FavoriteButton / WeatherWidget 等组件行为 CT

Phase 2
视觉治理

- 区分“基线更新”与“真实回归”
- 补多视口和关键 i18n 页面视觉覆盖

Phase 3
平台接入

- 接入 Azure PT 的 PR / main 执行路径
- 统一 Artifact、Trace、Portal 查看入口

Phase 4
智能审查

- 把 VLM 控制在视觉差异误报场景
- 以阈值、成本、置信度为约束上线

建议节奏：先解决业务高频回归与关键页面覆盖，再把治理能力逐步提升为平台标准。

RISKS & CONTROLS

主要风险与控制策略

技术负责人需要关注的是：这套方案如何避免失控，而不是它理论上能做多少。

定位器脆弱

尽量依赖 role / label / text ，必要时先经过 discovery 形成 locator strategy map 。

视觉误报

默认像素比对；只有用户显式开启时才用 VLM 做语义兜底，并加置信度阈值。

基线漂移

明确 authoritative baseline 环境，避免本地与 Azure 基线混用。

云上复杂度过早引入

把 Azure / CI / 门禁放到治理层，不污染日常测试链路。

业务覆盖失衡

用覆盖矩阵驱动补强优先级，先补高价值页面与旅程。

结果不可消费

输出必须带 HTML 报告、截图证据、JSON 或 Portal 结果，避免只留终端日志。

DECISION

建议的决策与下一步

这不是要不要做自动化的问题，而是要不要把现有能力升级成可持续的平台化体系。

决策 1

确认 ui-test 作为统一入口

统一团队使用口径，避免按人和工具分裂测试流程。

决策 2

优先补关键业务路径覆盖

优先级建议：Trip / Favorites 的功能型 E2E，其次补仅有视觉基线的组件行为 CT。

决策 3

治理能力按阶段启用

先把本地链路做稳定，再接入 Azure 门禁与 VLM；不要让治理能力反过来拖慢研发反馈。

最终结论：当前仓库已经具备“升级为平台化 UI 测试体系”的基础条件，建议以“覆盖补齐 + 治理延后”的方式推进。

APPENDIX A

附录：关键配置映射

把入口、配置、执行范围和治理边界映射清楚，便于技术评审时快速对齐责任。

配置 / 文件	定位	主要职责	关键边界
ui-test.agent.md	.github/agents	统一入口，负责把需求路由到 Discovery / CT / E2E / Visual / Governance 。	不直接承载完整实现逻辑，避免重新变回单体代理。
playwright.config.ts	仓库根目录	本地 E2E 与 Visual 默认配置，定义 baseURL 、reporter 、 webServer 、 project 拆分。	默认本地优先；Visual 仍以原生像素比对为主。
playwright-ct.config.ts	仓库根目录	组件测试配置，复用 Vite / PostCSS ，负责 CT 挂载环境。	CT 仅本地执行，不进入 Azure Playwright Workspace 。
playwright.service.config.ts	仓库根目录	Azure Playwright Workspace 云执行配置，增加 cloud reporter 与连接参数。	仅承接 E2E / Visual ；引入 Azure 凭据与平台侧并发。
tests/fixtures/visual-test.ts	tests/fixtures	封装像素优先、 VLM 兜底的视觉断言逻辑。	只有像素失败且 VLM_REVIEW=true 时才进入语义审查。

评审重点 1 ：是否接受“主流程本地优先、治理能力后置”的工程边界。

评审重点 2 ：是否确认 ui-test 与 3 套 Playwright 配置的职责拆分长期保持不变。

APPENDIX B

附录：当前覆盖矩阵与缺口

这页用于技术评审时回答“我们现在具体测到了哪里，还差哪里”。

对象	功能型 CT	E2E	视觉回归	结论
Home / Destinations / Detail / About	部分覆盖	已覆盖	已覆盖	核心主链路相对完整，可作为标准样板。
Favorites / Trip Planner / Trip Edit	未覆盖	弱覆盖 / 待补	已覆盖	属于当前最值得优先补强的业务缺口。
Navbar / DestinationCard / FilterBar / SearchBar	已覆盖	间接覆盖	部分覆盖	组件行为断言基础较好，可持续复用。
FavoriteButton / WeatherWidget / ScrollToTop / Layout	弱覆盖 / 未覆盖	间接覆盖	已覆盖	目前更偏视觉基线，缺行为型验证。
Azure / VLM 治理链路	不适用	按需启用	按需启用	能力已具雏形，但不应早于本地链路标准化。

优先级 A

先补业务高价值 E2E

Trip / Favorites / TripEdit 直接影响业务闭环，应优先补完功能流验证。

优先级 B

再补组件行为 CT

针对仅有视觉基线的组件补断言，降低 UI 变化误判与行为漏测。

优先级 C

最后补治理门禁

等本地链路、基线策略和覆盖矩阵稳定后，再引入 Azure / VLM 门禁。

APPENDIX C

附录：执行命令与治理触发条件

这页用于回答“团队如何跑、何时上云、何时启用 VLM”。

常用执行命令

场景	命令
本地 E2E	`npm run test:e2e`
本地 Visual	`npm run test:visual`
本地 CT	`npm run test:ct`
本地更新视觉基线	`npm run test:update-snapshots`
Azure E2E	`npm run test:azure:e2e`
Azure Visual	`npm run test:azure:visual`
启用 VLM 的本地 Visual	`npm run test:visual:vlm`
启用 VLM 的 Azure Visual	`npm run test:azure:visual:vlm`

治理触发条件

- 只有明确要求 PR 门禁、统一浏览器环境、平台报表时，才切到 Azure Playwright Workspace。
- 只有像素差异失败且团队希望降低视觉误报时，才显式开启 VLM_REVIEW=true。
- CT 永远留在本地，因为它依赖本地组件挂载和 Vite 开发上下文。
- 本地链路没有稳定之前，不建议先上云治理，否则会把问题复杂化而不是解决问题。

给技术负责人的控制点

先控制入口、配置、覆盖优先级，再控制云执行与语义审查成本。